`Too Many Requests` response. The resulting configuration in the application.yml file is shown in the following listing.

Listing 9.14 Configuring a request rate limiter as a gateway filter

```
spring:
  cloud:
    gateway:
      default-filters:
        name: RequestRateLimiter
          args:
            redis-rate-limiter:
              replenishRate: 10     ❶
              burstCapacity: 20     ❷
              requestedTokens: 1    ❸
```

❶ Number of tokens dripped in the bucket each second

❷ Allows request bursts of up to 20 requests

❸ How many tokens a request costs

There’s no general rule to follow in coming up with good numbers for the request rate limiter. You should start with your application requirements and go with a trial and error approach: analyze your production traffic, tune the configuration, and do this all over again until you achieve a setup that keeps your system available while not affecting the user experience badly. Even after that, you should keep monitoring the status of your rate limiters, since things can change in the future.

Spring Cloud Gateway relies on Redis to keep track of the number of requests happening each second. By default, each user is assigned a bucket. However, we haven’t introduced an authentication mechanism yet, so we’ll use a single bucket for all requests until we address the security concerns in chapters 11 and 12.

NOTE What happens if Redis becomes unavailable? Spring Cloud Gateway has been built with resilience in mind, so it will keep its service level, but the rate limiters would be disabled until Redis is up and running again.